

08

Rečnici

Parovi podataka, kao kodon i aminokiselina ili enzim i motiv, uz brzu pretragu po ključu

Od 4 baze do 64 trinukleotida

4 baze

```
a_count = dna.count("A")  
t_count = dna.count("T")  
g_count = dna.count("G")  
c_count = dna.count("C")
```

16 dinukleotida

```
aa_count = dna.count("AA")  
at_count = dna.count("AT")  
ag_count = dna.count("AG")  
# ... još 13 linija
```

64 trinukleotida

```
aaa_count = dna.count("AAA")  
aat_count = dna.count("AAT")  
aag_count = dna.count("AAG")  
# ... još 61 linija
```

Sekvenca od 20 baza ima samo 18 preklapajućih trinukleotida: bar 46 od 64 promenljive biće nula.

Pokušaj: svi brojevi u jednoj listi

```
dna = "AATGATCGATCGTACGCTGA"
all_counts = []
for base1 in ['A', 'T', 'G', 'C']:
    for base2 in ['A', 'T', 'G', 'C']:
        for base3 in ['A', 'T', 'G', 'C']:
            trinucleotide = base1 + base2 + base3
            count = dna.count(trinucleotide)
            all_counts.append(count)
print(all_counts)
```

```
[0, 1, 0, 0, 0, 0, 1, 2, 0, 0, ...]
```

Podaci su i dalje retki: 50 od 64 broja su nule.

Broj je odvojen od trinukleotida. Da je TGA 25. po redu, moramo sami da izračunamo:

```
print(all_counts[24]) # 2
```

Šta nam zapravo treba: parovi podataka

Promenljiva za svaki

`aaa_count`, `aat_count`, ...: 64 promenljive, a većina čuva nulu.

Lista brojeva

`[0, 1, 0, 0, ...]`: broj je odvojen od trinukleotida. Da je TGA na indeksu 24, moramo sami da znamo.

Dve paralelne liste

Moraju ostati usklađene, a `index()` traži element po element, što je sporo za velike liste.

Parovi ključ–vrednost su svuda

Ključ	Vrednost
trinukleotid	broj pojavljivanja
ime proteina	sekvenca
restrikcioni enzim	motiv
kodon	aminokiselina
uzorak	koordinate
reč	definicija

Rečnik: od ključa do vrednosti

```
enzymes = {  
    'EcoRI' : r'GAATTC',  
    'AvaII' : r'GG(A|T)CC',  
    'BisI'  : r'GC[ATGC]GC'  
}  
print(enzymes['BisI']) # GC[ATGC]GC  
# prazan rečnik, pa dodavanje stavki  
enzymes = {}  
enzymes['EcoRI'] = r'GAATTC'  
# pop vraća vrednost i briše ključ  
enzymes.pop('EcoRI')
```

Pravila

- Vitičaste zagrade; stavka je par **ključ: vrednost**, a stavke se razdvajaju zarezima
- Pristup: **rečnik[ključ]**, kao kod liste, ali sa ključem umesto indeksa
- Ključevi su stringovi ili brojevi i moraju biti jedinstveni
- Vrednosti mogu biti bilo kog tipa
- Dodavanje: **rečnik[ključ] = vrednost**

Brojanje trinukleotida: rečnik umesto liste

```
dna = "AATGATCGATCGTACGCTGA"
counts = {}
for base1 in ['A', 'T', 'G', 'C']:
    for base2 in ['A', 'T', 'G', 'C']:
        for base3 in ['A', 'T', 'G', 'C']:
            trinucleotide = base1 + base2 + base3
            count = dna.count(trinucleotide)
            counts[trinucleotide] = count

print(counts)
print(counts['TGA'])
```

```
{ 'AAA': 0, 'AAT': 1, 'AAG': 0, ... }
2
```

Trinukleotid i njegov broj su zajedno u jednoj promenljivoj: nema pamćenja redosleda ni usklađivanja dve liste.

Ali i dalje čuvamo 50 nula. Sledeći korak: samo brojevi veći od nule.

Brojanje trinukleotida rečnikom

```
dna = "AATGATCGATCGTACGCTGA"
counts = {}
for base1 in ['A', 'T', 'G', 'C']:
    for base2 in ['A', 'T', 'G', 'C']:
        for base3 in ['A', 'T', 'G', 'C']:
            trinucleotide = base1 + base2 + base3
            count = dna.count(trinucleotide)
            if count > 0:
                counts[trinucleotide] = count
```

```
print(counts['TGA'])           # 2
print(counts['AAA'])           # KeyError
print('AAA' in counts)         # False
print(counts.get('AAA', 0))    # 0
```

get(ključ, podrazumevano)

Vraća podrazumevanu vrednost kad ključa nema. Zato ne moramo da čuvamo nule niti da pre svakog pristupa proveravamo sa `in`.

Rečnik čuva samo trinukleotide koji se javljaju: {'AAT': 1, 'ATG': 1, 'ATC': 2, ...}

Prolazak kroz rečnik

PREKO KLJUČEVA: KEYS()

```
for trinucleotide in counts.keys():  
    if counts.get(trinucleotide) == 2:  
        print(trinucleotide)
```

PREKO PAROVA: ITEMS(), PREPORUČENO

```
for trinucleotide, count in counts.items():  
    if count == 2:  
        print(trinucleotide)
```

IZLAZ OBE PETLJE

ATC
TGA
TCG
GAT

Za sortiran redosled: `for trinucleotide in sorted(counts.keys()):`

Rečnik brzo nalazi vrednost za ključ, ali ne i obrnuto, kao i pravi rečnik.

Korak 1: deljenje sekvence na kodone

Ručno, isecanjem

```
dna = "ATGTTCCGGT"  
codon1 = dna[0:3]  
codon2 = dna[3:6]  
codon3 = dna[6:9]  
print(codon1, codon2, codon3)
```

ATG TTC GGT

Brojeve generiše range

```
for start in range(0, 7, 3):  
    codon = dna[start:start+3]  
    print("one codon is " + codon)
```

one codon is ATG
one codon is TTC
one codon is GGT

split ne pomaže: između kodona nema graničnika. Početak raste za 3, a kraj je početak + 3.

Korak 2: dokle ide range?

`range(0, 7, 3)` za kraću sekvencu "ATGTT":

'ATG', 'TT', ''

nepotpun i prazan kodon

`range(0, 7, 3)` za dužu sekvencu od 23 baze:

'ATG', 'TTC', 'GGT'

četvrti i kasniji kodoni se preskaču

```
dna = "ATGTTCCGGT"  
# početak poslednjeg kodona: još bar 2 baze iza  
last_codon_start = len(dna) - 2  
for start in range(0, last_codon_start, 3):  
    codon = dna[start:start+3]  
    print("one codon is " + codon)
```

Kraj opsega zavisi od dužine sekvence. Nepotpun kodon na kraju se ne prevodi.

Korak 3: genetski kod u rečniku

```
gencode = {  
    'ATA': 'I', 'ATC': 'I', 'ATT': 'I', 'ATG': 'M',  
    'ACA': 'T', 'ACC': 'T', 'ACG': 'T', 'ACT': 'T',  
    'AAC': 'N', 'AAT': 'N', 'AAA': 'K', 'AAG': 'K',  
    'AGC': 'S', 'AGT': 'S', 'AGA': 'R', 'AGG': 'R',  
    'CTA': 'L', 'CTC': 'L', 'CTG': 'L', 'CTT': 'L',  
    'CCA': 'P', 'CCC': 'P', 'CCG': 'P', 'CCT': 'P',  
    'CAC': 'H', 'CAT': 'H', 'CAA': 'Q', 'CAG': 'Q',  
    'CGA': 'R', 'CGC': 'R', 'CGG': 'R', 'CGT': 'R',
```

```
'GTA': 'V', 'GTC': 'V', 'GTG': 'V', 'GTT': 'V',  
'GCA': 'A', 'GCC': 'A', 'GCG': 'A', 'GCT': 'A',  
'GAC': 'D', 'GAT': 'D', 'GAA': 'E', 'GAG': 'E',  
'GGA': 'G', 'GGC': 'G', 'GGG': 'G', 'GGT': 'G',  
'TCA': 'S', 'TCC': 'S', 'TCG': 'S', 'TCT': 'S',  
'TTC': 'F', 'TTT': 'F', 'TTA': 'L', 'TTG': 'L',  
'TAC': 'Y', 'TAT': 'Y', 'TAA': '_', 'TAG': '_',  
'TGC': 'C', 'TGT': 'C', 'TGA': '_', 'TGG': 'W'}
```

```
print(gencode['CAT'])  
print(gencode.get('GTC'))
```

H
V

Kodoni su ključevi, aminokiseline vrednosti. Stop kodoni su označeni sa _.

Korak 4: protein raste kodon po kodon

```
dna = "ATGTTCCGGT"  
last_codon_start = len(dna) - 2  
protein = ""  
for start in range(0, last_codon_start, 3):  
    codon = dna[start:start+3]  
    aa = gencode.get(codon)  
    protein = protein + aa  
print("protein sequence is " + protein)
```

protein sequence is MFG

Kao ribozom: novu aminokiselinu dodajemo na kraj proteina koji raste.

Kod je koristan, pa ga pakujemo u funkciju `translate_dna(dna)` koja vraća protein.

Funkcija koristi `gencode` definisan van nje: svestan izuzetak od pravila iz pogl. 5.

Korak 5: testovi otkrivaju dve greške

```
print(translate_dna("ATGTTCCGGT"))  
print(translate_dna("ATCGATCGATCGTTGCTTATCGATCAG"))  
print(translate_dna("actgatcgtagcttgcttacgtatcgtat"))  
print(translate_dna("ACGATCGATCGTNACGTACGATCGTACTCG"))
```

```
MFG  
IDRSLLIDQ  
TypeError: can only concatenate str (not  
"NoneType") to str
```

Mala slova

Kodon 'act' nije ključ u rečniku, pa get vraća None. Rešenje:
`gencode.get(codon.upper())`

Nepoznata baza N

Kodona 'NAC' nema u rečniku. Nepoznata aminokiselina je X:
`gencode.get(codon.upper(), 'X')`

Konačno rešenje

```

gencode = {
    'ATA': 'I', 'ATC': 'I', 'ATT': 'I', 'ATG': 'M',
    # ... svih 64 kodona ...
    'TGC': 'C', 'TGT': 'C', 'TGA': '_', 'TGG': 'W'}

def translate_dna(dna):
    last_codon_start = len(dna) - 2
    protein = ""
    for start in range(0, last_codon_start, 3):
        codon = dna[start:start+3]
        aa = gencode.get(codon.upper(), 'X')
        protein = protein + aa
    return protein

```

DNK

ATGTTTCGGT

ATCGATCGATCGTTGCTTATCGATCAG

actgatcgtagcttgcttacgtatcgtat

ACGATCGATCGTNACGTACGATCGTACTCG

PROTEIN

MFG

IDRSLLIDQ

TDRSLLTYR

TIDRXVRSYS

- `len(dna) - 2`: poslednji kodon mora biti ceo
- `codon.upper()`: radi i za mala slova
- `get(..., 'X')`: kodon sa N postaje X

Rezime i vežba

Šta smo naučili

- Parovi ključ–vrednost u rečniku
- Pravljenje, pristup, dodavanje i pop
- in i get sa podrazumevanom vrednošću
- keys, items i sorted
- Ključevi su jedinstveni stringovi ili brojevi
- Brza pretraga po ključu, ali ne i obrnuto

Vežba: prevođenje DNK u protein

Po standardnom genetskom kodu (NCBI):

1. Podelite sekvencu na kodone: range sa korakom 3
2. Za svaki kodon nađite aminokiselinu u rečniku
3. Spojite aminokiseline u proteinsku sekvencu
4. Test primere pretvorite u assert naredbe